Vel Tech Multi Tech

Dr.Rangarajan Dr.Sakunthala Engineering College

An Autonomous Institution

(Approved by AICTE,New Delhi & Affiliated to Anna University,Chennai)

ISO 9001:2015 Certified Institution,
Accredited by NBA (BME, CSE, ECE, EEE, IT & MECH),
Accredited by NAAC with 'A' Grade with CGPA of 3.49,
#42, Avadi-Vel Tech Road, Avadi, Chennai-600062, Tamilnadu, India

DEPARTMENT OF INFORMATION TECHNOLOGY

LAB MANUAL

COURSE CODE : 231CS52B

COURSE NAME : EMBEDDED SYSTEMS AND IOT LABORATORY

BRANCH : INFORMATION TECHNOLOGY

YEAR : III

SEMESTER : VI

Prepared by
Mrs. S.Sri Nandhini Kowsalya., M.E.,
Mrs.M.Uma Latha., M.Tech.,

DEPARTMENT OF INFORMATION TECHNOLOGY

VISION

- To emerge as centre for academic eminence in the field of information technology through innovative learning practices.

MISSION

- M1-To provide good teaching and learning environment for quality education in the field of information technology.
- M2-To propagate lifelong learning.
- M3-To impart the right proportion of knowledge, attitudes and ethics in students to enable them take up positions of responsibility in the society and make significant contributions.

Vel Tech Multi Tech

Dr.Rangarajan Dr.Sakunthala Engineering College

An Autonomous Institution

(Approved by AICTE,New Delhi & Affiliated to Anna University,Chennai)

ISO 9001:2015 Certified Institution,
Accredited by NBA (BME, CSE, ECE, EEE, IT & MECH),
Accredited by NAAC with 'A' Grade with CGPA of 3.49,
#42, Avadi-Vel Tech Road, Avadi, Chennai-600062, Tamilnadu, India

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

Name: _____

Year: _____ **Semester:** _____ **Programme:** B.Tech – Information Technology,

University Register No: _____ **College Roll No:** _____

Certified that this is the bonafide record of work done by the above student in **231CS52B-**

EMBEDDED SYSTEMS AND IOT LABORATORY during the academic year 2025-26.

Signature of Course In charge **Signature of Head of the Department**

Submitted for the University Practical Examination held on.....at VEL TECH MULTITECH Dr RANGARAJAN Dr SAKUNTHALA ENGINEERING COLLEGE, #42, AVADI- VEL TECH ROAD, AVADI, CHENNAI-600062.

Signature of Examiners

Internal Examiner……... External Examiner……

Date…….

DEPARTMENT OF INFORMATION TECHNOLOGY

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

PEOs	PROGRAMME EDUCATIONAL OBJECTIVES(PEOs)
PEO1	Graduates will demonstrate technical competency and leadership skills to lead a successful career in the field of IT&ITES.
PEO2	Graduate will exhibit a commitment to communicate effectively in diverse environment and apply proficiency towards societal issue with human values.
PEO3	Graduates will pursue lifelong learning in generating innovative solutions to the changing industrial needs using research and problem-solving skills.

PROGRAMME SPECIFIC OUTCOMES (PSOs)

PSO's	PROGRAMME SPECIFIC OUTCOMES(PSOs)
PSO1	An ability to apply design and development principles in the construction of software systems of varying complexity.
PSO2	The use of current application software, the design and use of operating systems and the analysis, design, testing and documentation of computer programs for the use in the information engineering technologies.
PSO3	The design techniques, analysis and the building, testing, operation and maintenance of networks, databases, security and computer systems. (both hardware and software).

PROGRAMME OUTCOMES (POs)

PO's	PROGRAMME OUTCOMES
PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4).
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems / components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5).
PO4	Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6).
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9).
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective Presentations considering cultural, language, and learning differences.
PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) Independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Knowledge and Attitude Profiles (WK)	
WK1: Understanding of natural and social sciences.	**WK2**: Mathematics, numerical analysis, data analysis, and computing.
WK3: Engineering fundamentals.	**WK4**: Specialized engineering knowledge.
WK5: Engineering design and operations, including sustainability.	**WK6**: Engineering practice (technology).
WK7: Role of engineering in society, sustainability, and professional responsibility.	**WK8**: Current research literature and critical thinking.
WK9: Ethics, professional responsibilities, and inclusive behavior.	

COURSE CODE: 231CS52B
COURSE NAME: EMBEDDED SYSTEMSAND IOT LABORATORY

COURSE OBJECTIVES:

- To learn the internal architecture and programming of an embedded processor
- To build a small low-cost embedded and IoT system using Arduino/ RaspberryPi/ Open platform.
- To apply the concept to Internet to Things in real world scenario.

SYLLABUS

LIST OF EXERCISES
1. Write 8051 Assembly Language experiments using simulator and Test data transfer between registers and memory.
2. Perform ALU operations and Write Basic and arithmetic Programs Using Embedded C.
3. Introduction to Arduino platform and programming.
4. Explore different communication methods with IoT devices(Zigbee, GSM, Bluetooth).
5. Introduction to Raspberry PI platform and python programming.
6. Interfacing sensors with Raspberry PI.
7. Communicate between Arduino and Raspberry PI using any wireless medium.
8. Setup a cloud platform to log the data and Log Data using Raspberry PI and upload to the cloud platform.
9. Design an IOT based system
10. Mini Project.

COURSE OUTCOMES

At the end of the course, the student should be able to

Course Outcome	CO Statements
CO1	Apply 8051 and Arduino microcontrollers for data transfer and logical operations using Assembly and Embedded C
CO2	Develop and test embedded applications with Arduino and Raspberry Pi using sensors and wireless technologies.
CO3	Design IoT applications using Arduino/Raspberry Pi /open platform.

CO-PO, CO-PSO MAPPING

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	3	3	2	-	2	-	-	-	-	-	-	3	2	-
CO2	3	3	3	1	3	-	-	-	-	-	-	3	3	-
CO3	3	3	3	2	3	-	-	1	-	-	-	3	3	2
Average	3	3	3	2	3	-	-	1	-	-	-	3	3	2

INDEX

Ex.No	Date	Name of the Experiment	CO	Page No	Marks	Faculty Sign
1		Write 8051 Assembly Language experiments using simulator and Test data transfer between registers and memory.	CO1			
2		(A) Perform ALU operations (B) Write Basic and arithmetic Programs Using Embedded C.	CO1			
3		Introduction to Arduino platform and programming	CO1			
4		Explore different communication methods with IoT devices	CO2			
5		Introduction to Raspberry PI platform and python programming	CO2			
6		Interfacing sensors with Raspberry PI	CO2			
7		Communicate between Arduino and Raspberry PI using any wireless medium	CO2			
8		Setup a cloud platform to log the data and Log Data using Raspberry PI and upload to the cloud platform	CO3			
9		Design an IOT based system	CO3			
10		Mini Project	CO3			

Ex.No:1	Write 8051 Assembly Language experiments using simulator and Test data transfer between registers and memory.
Date:	

AIM:

To write and execute an 8051 Assembly Language Program for 8-bit addition and to transfer data between registers and memory using the Keil simulator.

SOFTWARE REQUIRED:

S.No	Software Requirements	Quantity
1	Keilμvision5 IDE	1

PROCEDURE:

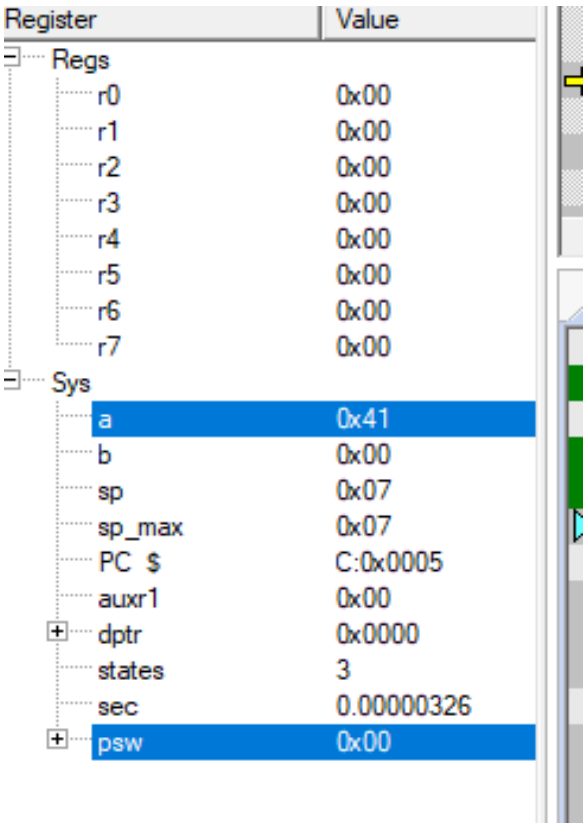
1. Launch Keil μVision. If any project is already open, go to Project → Close Project to start fresh.
2. Select Project → New μVision Project, assign a suitable name and location for your project.
3. Choose the appropriate microcontroller (e.g., AT89C51ED2, AT89C51, or AT89C52) from the device list.
4. When prompted, add the startup file. Then go to File → New to open a new editor window for writing code.
5. Write the assembly language program in the editor window. Save the file with a .asm extension.
6. Add the newly created .asm file to the project group.
7. Click Build Target (or press F7) to assemble the program and check for any syntax errors.
8. Enter Debug Mode, then click Run or Step Run to execute the program. Observe the results in the simulator windows such as Registers, Memory, and Output.

PROGRAM:

8-bit addition

```
ORG 0000H

CLR C

MOV A, #20H

ADD A, #21H

MOV R0, A

END
```

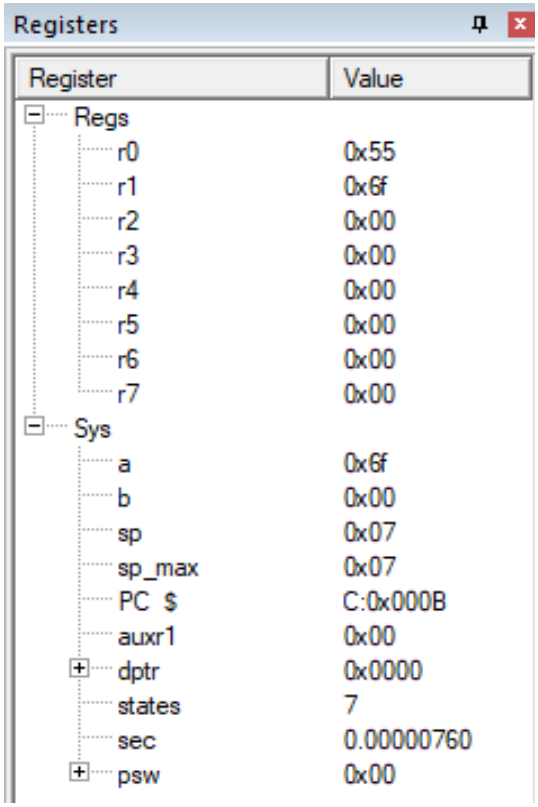
SAMPLE OUTPUT:



Register to Memory Data Transfer

```
ORG 0000H
CLR C
MOV R0, #55H
MOV R1, #6FH
MOV A, R0
MOV 30H, A
MOV A, R1
MOV 31H, A
END
```

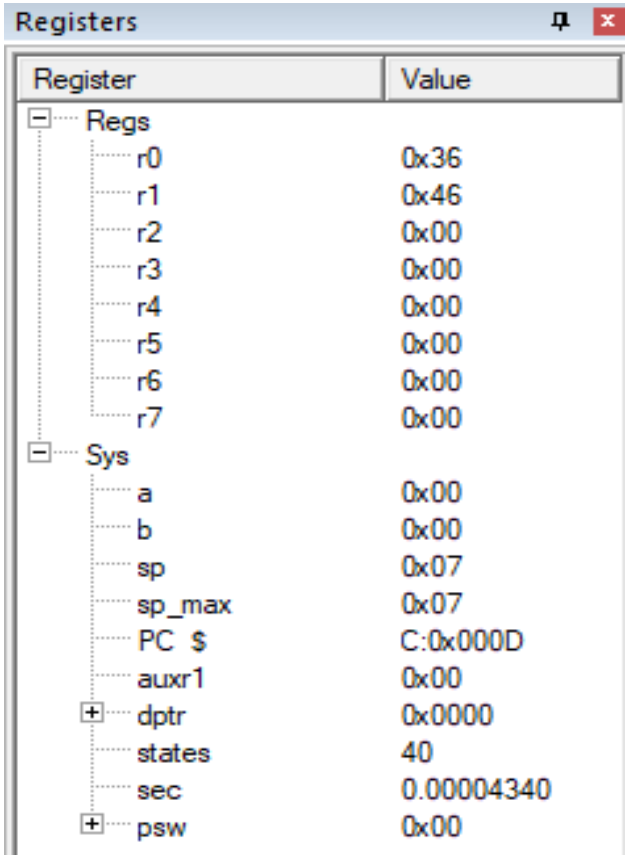
SAMPLE OUTPUT:



Memory-to-Memory Data Transfer

```
ORG 0000H
CLR C
MOV R0, #30H
MOV R1, #40H
MOV R7, #06H
BACK: MOV A, @R0
MOV @R1, A
INC R0
INC R1
DJNZ R7, BACK
END
```

SAMPLE OUTPUT:



RESULT:

Thus the program 8051 Assembly Language experiments using simulator was written and executed successfully.

Ex.No:2.A	Perform ALU operation
Date:	

AIM:

To write and execute the ALU program using the Keil simulator.

SOFTWARE TOOLS REQUIRED:

S.No	Software Requirements	Quantity
1	Keilμvision5 IDE	1

PROCEDURE:

1. Launch Keil μVision. If any project is already open, go to Project → Close Project to start fresh.
2. Select Project → New μVision Project, assign a suitable name and location for your project.
3. Choose the appropriate microcontroller (e.g., AT89C51ED2, AT89C51, or AT89C52) from the device list.
4. When prompted, add the startup file. Then go to File → New to open a new editor window for writing code.
5. Write the assembly language program in the editor window. Save the file with a .asm extension.
6. Add the newly created .asm file to the project group.
7. Click Build Target (or press F7) to assemble the program and check for any syntax errors.
8. Enter Debug Mode, then click Run or Step Run to execute the program. Observe the results in the simulator windows such as Registers, Memory, and Output.

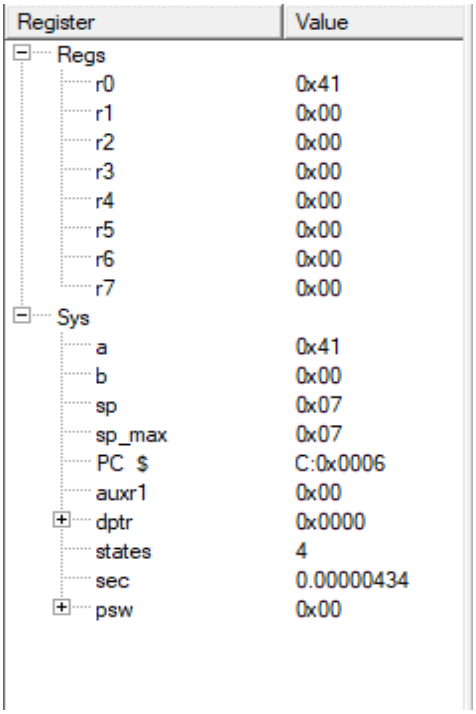
PROGRAM:

```
ORG 0000H
; Addition
MOV A, 20H
ADD A, 21H
MOV 41H, A
; Subtraction
MOV A, #20H
SUBB A, #18H
MOV 42H, A
; Multiplication
MOV A, #03H
MOV B, #04H
MUL AB
MOV 43H, A
; Division
MOV A, #95H
```

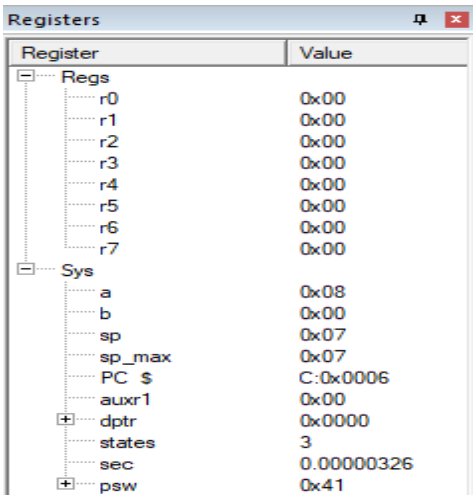
```
MOV B, #10H
DIV AB
MOV 44H, A
MOV 45H, B
; AND Operation
MOV A, #25H
MOV B, #12H
ANL A, B
MOV 46H, A
; OR Operation
MOV A, #25H
MOV B, #15H
ORL A, B
MOV 47H, A
; XOR Operation
MOV A, #45H
MOV B, #67H
XRL A, B
MOV 48H, A
; NOT Operation
MOV A, #45H
CPL A
MOV 49H, A
END
```

SAMPLE OUTPUT:

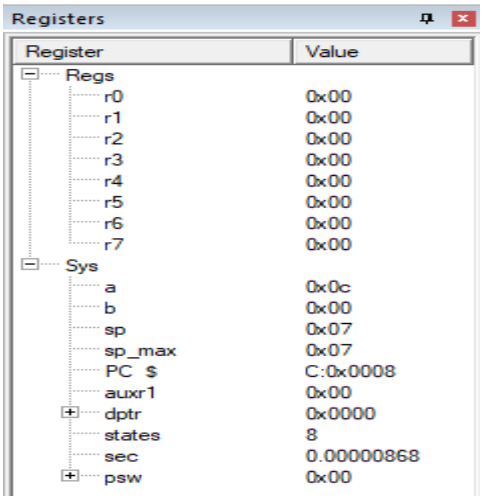
Addition:



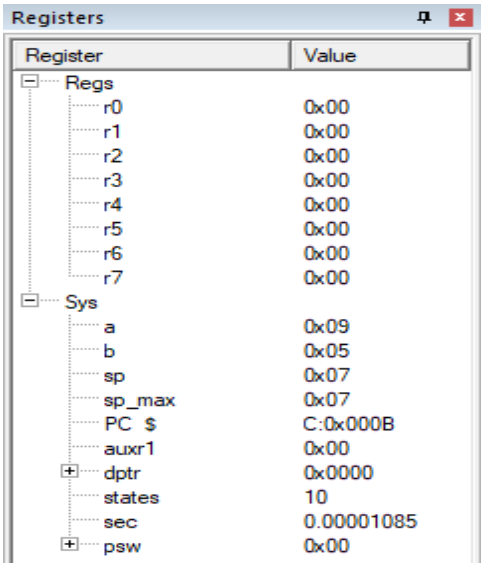
Subtraction:



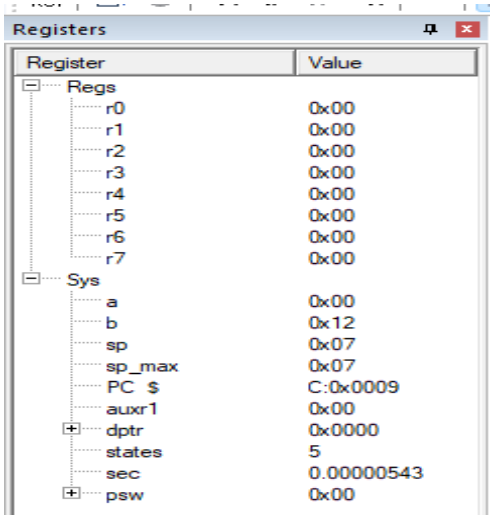
Multiplication:



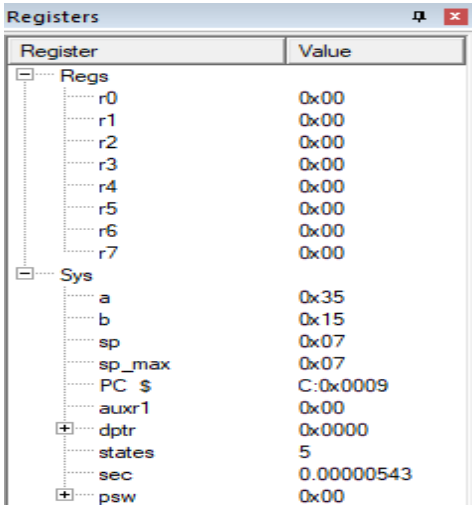
Division:



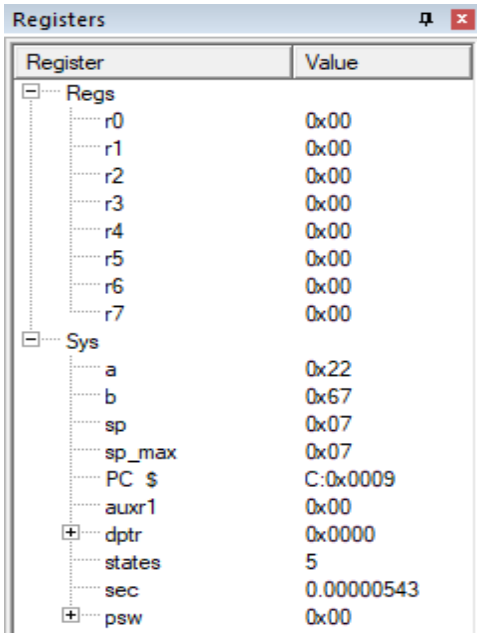
AND:


Registers
Register
Value
Regs
r0 0x00
r1 0x00
r2 0x00
r3 0x00
r4 0x00
r5 0x00
r6 0x00
r7 0x00
Sys
a 0x00
b 0x12
sp 0x07
sp_max 0x07
PC $ C:0x0009
auxr1 0x00
dptr 0x0000
states 5
sec 0.00000543
psw 0x00

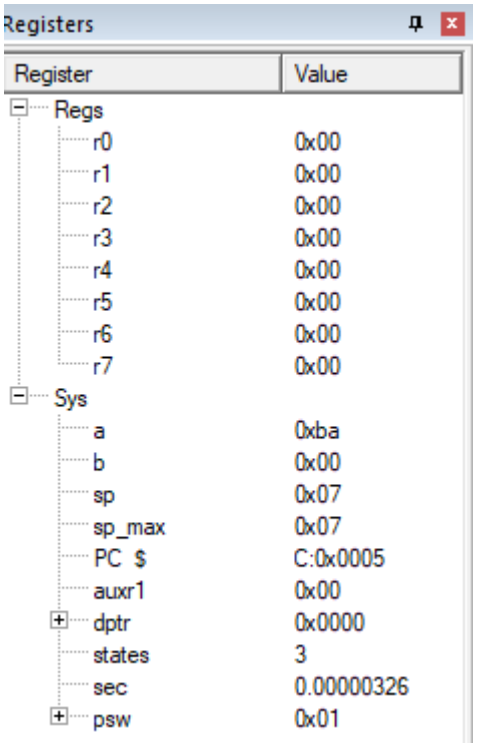
OR:


Registers
Register
Value
Regs
r0 0x00
r1 0x00
r2 0x00
r3 0x00
r4 0x00
r5 0x00
r6 0x00
r7 0x00
Sys
a 0x35
b 0x15
sp 0x07
sp_max 0x07
PC $ C:0x0009
auxr1 0x00
dptr 0x0000
states 5
sec 0.00000543
psw 0x00

XOR:


Registers
Register
Value
Regs
r0 0x00
r1 0x00
r2 0x00
r3 0x00
r4 0x00
r5 0x00
r6 0x00
r7 0x00
Sys
a 0x22
b 0x67
sp 0x07
sp_max 0x07
PC $ C:0x0009
auxr1 0x00
dptr 0x0000
states 5
sec 0.00000543
psw 0x00

NOT:



RESULT

Thus the program to perform ALU operation was written and executed successfully.

Ex.No:2.B	Write Basic and arithmetic Programs Using Embedded C.
Date:	

AIM:

To write a basic and arithmetic programs in embedded C program to control a port 0 pin 0 connected to an 8051microcontroller using a Keil simulator.

SOFTWARE REQUIRED:

S.No	Software Requirements	Quantity
1	Keilμvision5 IDE	1

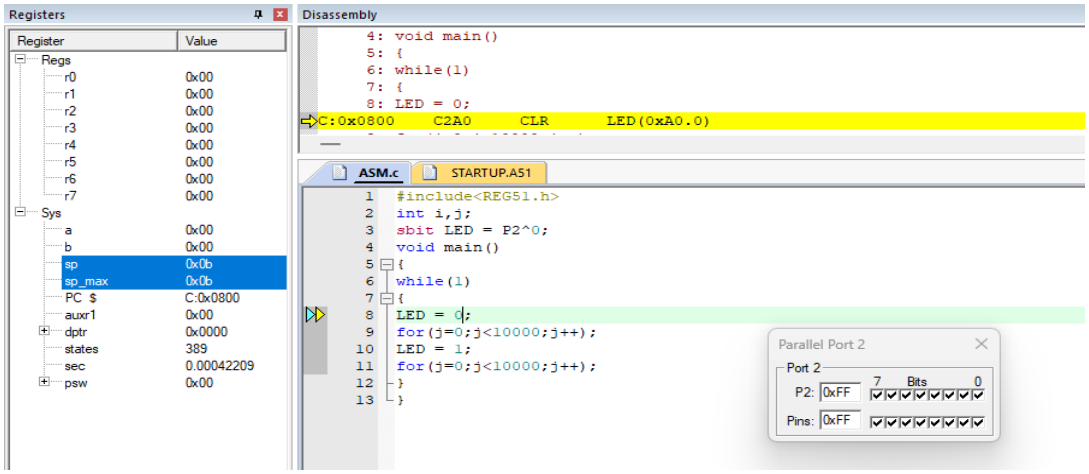
PROCEDURE:

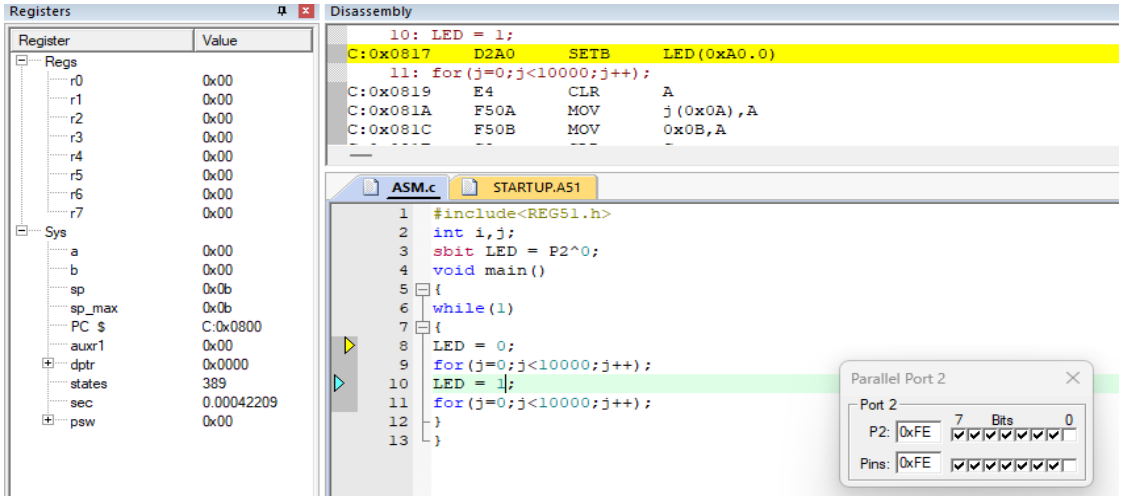
1. Launch Keil μVision. If any project is already open, go to Project → Close Project to start fresh.
2. Select Project → New μVision Project, assign a suitable name and location for your project.
3. Choose the appropriate microcontroller (e.g., AT89C51ED2, AT89C51, or AT89C52) from the device list.
4. When prompted, add the startup file. Then go to File → New to open a new editor window for writing code.
5. Write the assembly language program in the editor window. Save the file with a .C extension.
6. Add the newly created .C file to the project group.
7. Click Build Target (or press F7) to assemble the program and check for any syntax errors.
8. Enter Debug Mode, then click Run or Step Run to execute the program. Observe the results in the simulator windows such as Registers, Memory, and Output.

PROGRAM:

```
#include <REG51.h>
sbit LED = P2^0;
void main(void)
{
  unsigned int j;
  while(1)
  {
    LED = 0;
    for(j = 0; j < 10000; j++);
    LED = 1;
    for(j = 0; j < 10000; j++);      }
}
```

SAMPLE OUTPUT:

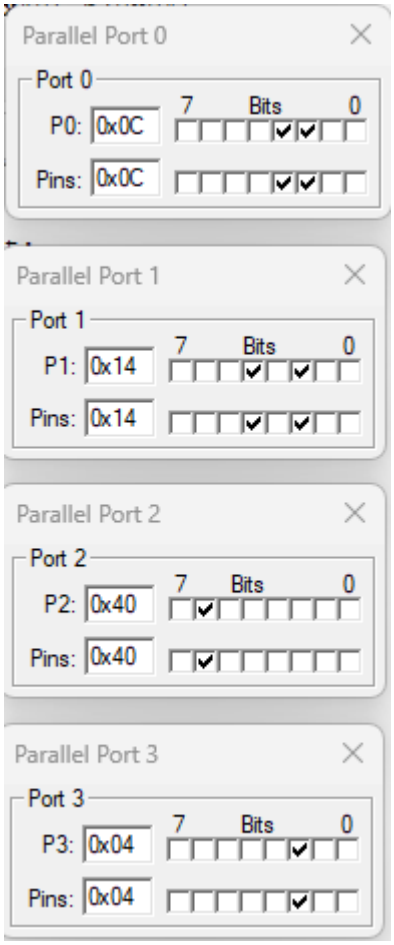




PROGRAM:

```
#include<REG51.H>
unsigned char a, b;
void main()
{
a=0x10;
b=0x04;
P0=a-b;
P1=a+b;
P2=a*b;
P3=a/b;
while(1);
}
```

SAMPLE OUTPUT:



RESULT

Thus the program of basic and arithmetic programs in embedded C program was written and executed successfully.

Ex.No:3	Introduction to Arduino platform and programming
Date:	

AIM:

To understand the Arduino platform and write programs to handle digital and analog signals, as well as perform serial communication.

HARDWARE & SOFTWARE TOOLS REQUIRED:

S.No	Hardware & Software Requirements	Quantity
1	Arduino IDE 2.0	1
2	Arduino UNO Development Board	1
3	Jumper Wires	few
4	Arduino USB Cable	1
5	Joystick Module	1

INTRODUCTION TO ARDUINO:

Arduino is a project, open-source hardware, and software platform used to design and build electronic devices. It designs and manufactures microcontroller kits and single-board interfaces for building electronics projects. The Arduino boards were initially created to help students with the non-technical background. The designs of Arduino boards use a variety of controllers and microprocessors. Arduino is an easy-to-use open platform for creating electronic projects. Arduino boards play a vital role in creating different projects. It makes electronics accessible to non-engineers, hobbyists, etc.

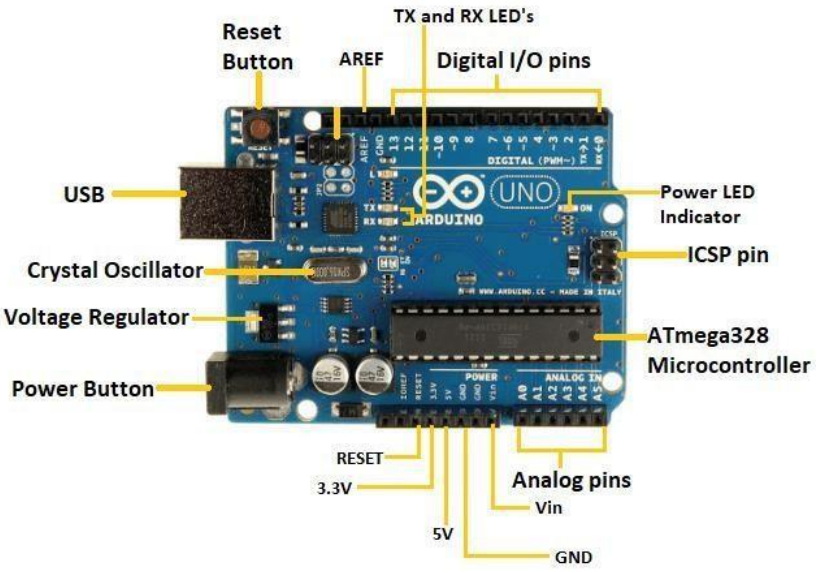
The various components present on the Arduino boards are a Microcontroller, Digital Input/output pins, USB Interface and Connector, Analog Pins, reset buttons, Power buttons, LEDs, Crystal oscillators, and Voltage regulators. Some components may differ depending on the type of board. The most standard and popular board used over time is Arduino UNO. The ATmega328 Microcontroller present on the UNO board makes it rather powerful than other boards. There are various types of Arduino boards used for different purposes and projects. The Arduino Boards are organized using the Arduino (IDE), which can run on various platforms. Here, IDE stands for Integrated Development Environment. Let's discuss some common and best Arduino boards.

INTRODUCTION TO ARDUINO UNO:

The Arduino UNO is a standard board of Arduino. Here UNO means 'one' in Italian. It was named UNO to label the first release of Arduino Software. It was also the first USB board released by Arduino. It is considered a powerful board used in various projects. Arduino. cc developed the Arduino UNO board. Arduino UNO is based on an ATmega328P microcontroller. It is easy to use compared to other boards, such as the Arduino Mega board, etc. The board consists of digital and analog Input/Output pins (I/O), shields, and other circuits. The Arduino UNO includes 6 analog pin inputs, 14 digital pins, a USB connector, a power jack, and an ICSP (In-Circuit Serial Programming) header. It is programmed based on IDE, which stands for Integrated Development Environment. It can run on both online and offline platforms. The IDE is common to all available boards of Arduino.

The Arduino board is shown below:

The components of Arduino UNO board are shown below:



Let's discuss each component in detail.

- ATmega328 Microcontroller- It is a single-chip Microcontroller of the ATmel family. The processor code inside it is of 8-bit. It combines Memory (SRAM, EEPROM, and Flash), Analog to Digital Converter, SPI serial ports, I/O lines, registers, timers, external and internal interrupts, and oscillator.
- ICSP pin - The In-Circuit Serial Programming pin allows the user to program using the firmware of the Arduino board.
- Power LED Indicator- The ON status of the LED shows the power is activated. When the power is OFF, the LED will not light up.
- Digital I/O pins- The digital pins have the value HIGH or LOW. The pins numbered from D0 to D13 are digital pins.
- TX and RX LED's- The successful flow of data is represented by the lighting of these LED's.
- AREF- The Analog Reference (AREF) pin is used to feed a reference voltage to the Arduino UNO board from the external power supply.
- Reset button- It is used to add a Reset button to the connection.
- USB- It allows the board to connect to the computer. It is essential for the programming of the Arduino UNO board.
- Crystal Oscillator- The Crystal oscillator has a frequency of 16MHz, which makes the Arduino UNO a powerful board.
- Voltage Regulator- The voltage regulator converts the input voltage to 5V.
- GND- Ground pins. The ground pin acts as a pin with zero voltage.
- Vin- It is the input voltage.
- Analog Pins- The pins numbered from A0 to A5 are analog pins. The function of Analog pins is to read the analog sensor used in the connection. It can also act as GPIO (General Purpose Input Output) pin.

TECHNICAL SPECIFICATIONS OF ARDUINO UNO

The technical specifications of the Arduino UNO are listed below:

- There are 20 Input/Output pins present on the Arduino UNO board. These 20 pins include 6 PWM pins, 6 analog pins, and 8 digital I/O pins.
- The PWM pins are Pulse Width Modulation capable.
- The crystal oscillator present in Arduino UNO comes with a frequency of 16MHz.
- It also has an Arduino-integrated WIFI module. Such Arduino UNO board is based on the Integrated WIFI ESP8266 Module and ATmega328P microcontroller.
- The input voltage of the UNO board varies from 7V to 20V.
- Arduino UNO automatically draws power from the external power supply. It can also draw power from the USB.

INTRODUCTION TO ARDUINO IDE 2.0:

The Arduino IDE 2.0 is an open-source project, currently in its beta-phase. It is a big step from it's sturdy predecessor, Arduino IDE 2.0, and comes with revamped UI, improved board & library manger, autocomplete feature and much more.

In this tutorial, we will go through step by step, how to download and install the software.

Download the editor

Downloading the Arduino IDE 2.0 is done through the Arduino Software page. Here you will also find information on the other editors available to use.

Requirements

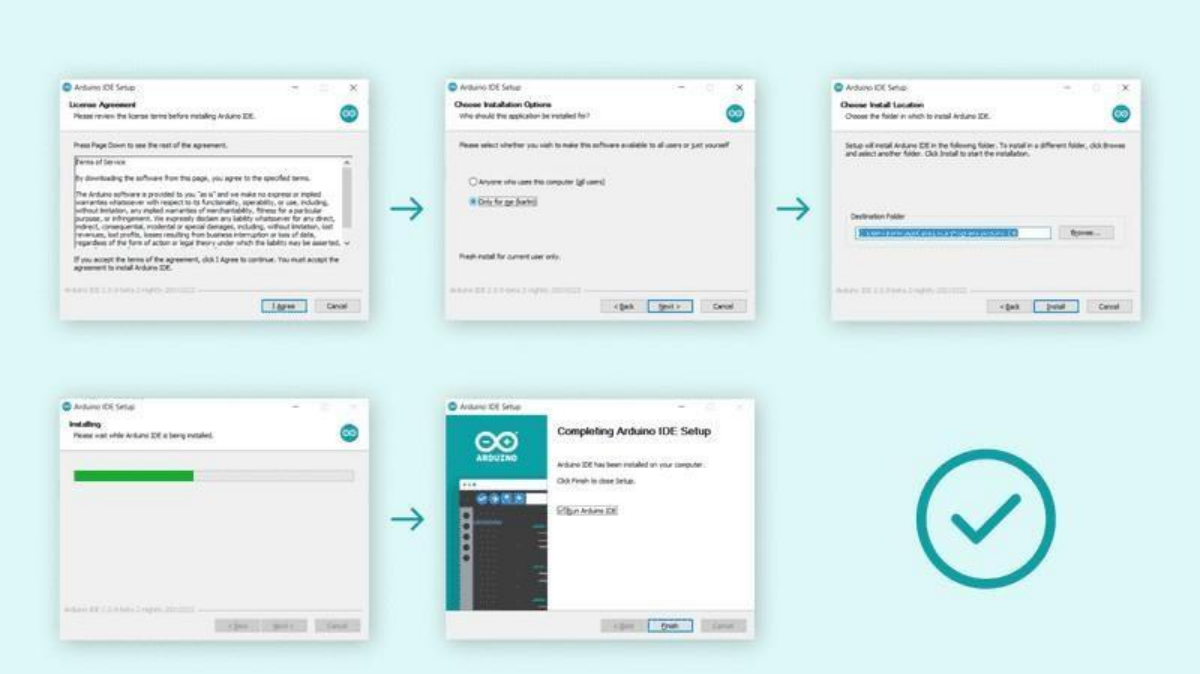
- Windows - Win 10 and newer, 64 bits
- Linux - 64 bits
- Mac OS X - Version 10.14: "Mojave" or newer, 64 bits

Installation

Windows

Download URL: https://www.arduino.cc/en/software

To install the Arduino IDE 2.0 on a Windows computer, simply run the file downloaded from the software page. Follow the instructions in the installation guide. The installation may take several minutes.



How to use the board manager with the Arduino IDE 2.0

The board manager is a great tool for installing the necessary cores to use your Arduino boards. In this quick tutorial, we will take a look at how to install one, and choosing the right core for your board!

Requirements

- Arduino IDE 2.0 installed.

Why use the board manager?

The board manager is a tool that is used to install different cores on your local computer. So what is a core, and why is it necessary that I install one?

Simply explained, a core is written and designed for specific microcontrollers. As Arduino have several different types of boards, they also have different type of microcontrollers.

For example, an Arduino UNO has an ATmega328P, which uses the AVR core, while an Arduino Nano 33 IoT has a SAMD21 microcontroller, where we need to use the SAMD core.

In conclusion, to use a specific board, we need to install a specific core.

Installing a Core

Installing a core in the Arduino IDE 2.0 is quick and straightforward. Follow the steps below:

1. Open the Arduino IDE 2.0.
2. In the left sidebar, you will see several icons. Click on the “Board Manager” icon (the computer chip symbol).
3. A list of available cores will appear.
    - For example, if you are using an Arduino Nano 33 IoT board, type the board name in the search field.
    - The correct core (SAMD) will appear, showing that it supports the Nano 33 IoT.
    - Click the “Install” button next to it.
4. The installation process will begin. It may take a few minutes, depending on your internet speed.
5. Once the installation is complete, the status will change to “INSTALLED” in the Boards Manager.

How to upload a sketch with the Arduino IDE 2.0

In the Arduino environment, we write **sketches** that can be uploaded to Arduino boards. In this tutorial, we will go through how to select a board connected to your computer, and how to upload a sketch to that board, using the Arduino IDE 2.0.

Requirements

- Arduino IDE 2.0 installed.

Verify VS Upload

There are two main tools when uploading a sketch to a board: verify and upload. The verify tool simply goes through your sketch, checks for errors and compiles it. The upload tool does the same, but when it finishes compiling the code, it also uploads it to the board.

A good practice is to use the verifying tool before attempting to upload anything. This is a quick way of spotting any errors in your code, so you can fix them before actually uploading the code. Uploading a sketch

Installing a core is quick and easy, but let's take a look at what we need to do.

1. Open the Arduino IDE 2.0.
2. With the editor open, let's take a look at the navigation bar at the top. At the very left, there is a checkmark and an arrow pointing right. The checkmark is used to verify, and the arrow is used to upload.
3. Click on the verify tool (checkmark). Since we are verifying an empty sketch, we can be sure it is going to compile. After a few seconds, we can see the result of the action in the console (black box in the bottom).
4. Now we know that our code is compiled, and that it is working. Now, before we can upload the code to our board, we will first need to select the board that we are using. We can do this by navigating to Tools > Port > {Board}. The board(s) that are connected to your computer should appear here, and we need to select it by clicking it. In this case, our board is displayed as COM44 (Arduino UNO).
5. With the board selected, we are good to go! Click on the upload button, and it will start uploading the sketch to the board.
6. When it is finished, it will notify you in the console log. Of course, sometimes there are some complications when uploading, and these errors will be listed here as well.

DIGITAL WRITE (LED BLINK)

CONNECTION:

Arduino UNO Pin	Arduino Development Board
2	**LED**

PROCEDURE:

1. Connect the LED to pin 2 of Arduino (anode to pin 2, cathode to GND via resistor).
2. Open Arduino IDE and create a new sketch.
3. Write the digital write program.
4. Select Arduino UNO board and correct COM port.
5. Upload the program to the board.
6. Observe the LED blinking at 1-second intervals.

PROGRAM:

```
void setup() {
  pinMode(2, OUTPUT);
}
void loop() {
  digitalWrite(2, HIGH);
  delay(1000);
  digitalWrite(2, LOW);
  delay(1000);
}
```

SAMPLE OUTPUT:

LED blinks continuously with 1-second ON/OFF delay.

DIGITAL READ (SWITCH CONTROL)
CONNECTION:

Arduino UNO Pin	Arduino Development Board
2	**LED**
5	**S1 (SW 1)**

PROCEDURE:

1. Connect an LED to pin 2 (anode to pin, cathode to GND).
2. Connect a push button to pin 5 with INPUT_PULLUP configuration.
3. Open Arduino IDE and write the digital read program.
4. Upload the program.
5. Press the switch to see LED blinking 5 times; LED stays OFF when switch is released.

PROGRAM:

```
void setup() {
  pinMode(2, OUTPUT);
  pinMode(5, INPUT_PULLUP);
}
```

```
void loop() {
  int sw = digitalRead(5);
  if(sw == 1) {
    for(int i = 0; i < 5; i++) {
      digitalWrite(2, HIGH);
      delay(1000);
      digitalWrite(2, LOW);
      delay(1000);
    }
  } else {
    digitalWrite(2, LOW);
  }
}
```

SAMPLE OUTPUT:

LED blinks 5 times when switch pressed, OFF otherwise.

ANALOG READ

CONNECTION:

Arduino UNO Pin	Arduino Development Board	Joystick Module
2	**LED**	
VCC or 5V		**+5V**
GND		**GND**
A0		**VRx or VRy**

PROCEDURE:

1. Connect joystick module to Arduino as per the table.
2. Connect LED to pin 2.
3. Open Arduino IDE and write the analog read program.
4. Upload the program.
5. Open Serial Monitor to view analog values.
6. Move joystick; LED turns ON if joystick value > 800, OFF otherwise.

PROGRAM:

```
void setup() {
  pinMode(2, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  int joystick = analogRead(A0);
  Serial.println(joystick);
  if(joystick > 800) digitalWrite(2, HIGH);
  else digitalWrite(2, LOW);
  delay(500);
}
```

SAMPLE OUTPUT:

Serial Monitor displays analog values (0–1023).

LED turns ON when joystick pushed above threshold.

ANALOG WRITE:

CONNECTION:

Arduino UNO Pin	Arduino Development Board
3	**LED**

PWM Pins: 3, 5, 6, 9, 10, 11

PROCEDURE:

1. Connect LED to pin 3 (PWM pin).
2. Open Arduino IDE and write the analog write (PWM) program.
3. Upload the program.
4. Observe LED gradually increasing and decreasing in brightness.

PROGRAM:

```
void setup() {
  pinMode(3, OUTPUT);
}
void loop() {
  for(int i = 0; i < 256; i++) {
    analogWrite(3, i);
    delay(20);
  }
  for(int i = 255; i >= 0; i--) {
    analogWrite(3, i);
    delay(20);
  }
}
```

SAMPLE OUTPUT:

LED brightness smoothly increases and decreases repeatedly.

SERIAL COMMUNICATION

CONNECTION:

Arduino UNO Pin	Arduino Development Board
4	**LED**

PROCEDURE:

1. Connect LED to pin 4.
2. Open Arduino IDE and write the serial communication program.
3. Upload the program.
4. Open Serial Monitor, send '1' to turn LED ON and '2' to turn LED OFF.

PROGRAM:

```
void setup() {
  Serial.begin(9600);
  pinMode(4, OUTPUT);
}
void loop() {
  if(Serial.available() > 0) {
  char data = Serial.read();
  Serial.println(data);
  if(data == '1') {
      digitalWrite(4, HIGH);
    } else if(data == '2') {
      digitalWrite(4, LOW);
    }
  }
}
```

SAMPLE OUTPUT:

LED turns ON or OFF based on Serial Monitor input.

RESULT:

Thus, the Programs for digital and analog signal control and serial communication were successfully written and executed in Embedded C using Arduino

Ex.No:4	Explore different communication methods with IoT devices
Date:	

AIM:

To explore different communication methods used with IoT devices.

HARDWARE & SOFTWARE TOOLS REQUIRED:

S.No	Hardware & Software Requirements	Quantity
1	Arduino IDE 2.0	1
2	Arduino UNO Development Board	2
3	Jumper Wires	few
4	Arduino USB Cable	3
5	HC-05 Bluetooth Module	2
6	Zigbee Module	2

BLUETOOTH COMMUNICATION

PROCEDURE:

1. Connect HC-05 Bluetooth module to Arduino as follows: VCC → 5V, GND → GND, TX → Pin 2, RX → Pin 3.
2. Connect an LED to Pin 4 of Arduino (with a current-limiting resistor if required).
3. Open Arduino IDE and write the Bluetooth control program (see below).
4. Upload the program to Arduino UNO.
5. Pair HC-05 module with a Bluetooth-enabled device (smartphone or PC).
6. Open a Serial Bluetooth terminal on the device.
7. Send '1' to turn the LED ON and '2' to turn it OFF.
8. Observe LED behavior to verify successful communication.

CONNECTIONS:

Arduino UNO Pin	Bluetooth Module	Arduino Development Board
VCC	**5V**	**-**
GND	**GND**	**-**
2	**Tx**	**-**
3	**Rx**	**-**
4	**-**	**LED**

PROGRAM:

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2,3); // RX, TX
void setup() {
  mySerial.begin(9600);
  Serial.begin(9600);
  pinMode(4, OUTPUT);
}void loop() {
```

```
  if(mySerial.available() > 0) {
    char data = mySerial.read();
    Serial.println(data);
    if(data == '1') {
      digitalWrite(4, HIGH);
      Serial.println("LED ON");
    } else if(data == '2') {
      digitalWrite(4, LOW);
      Serial.println("LED OFF");
    }
  }
}
```

SAMPLE OUTPUT:

LED turns ON when '1' is sent.

` LED turns OFF when '2' is sent.

PROCEDURE:

Zigbee Communication Transmitter Side:

1. Connect Zigbee transmitter module: VCC → 5V, GND → GND, TX → Pin 2, RX → Pin 3.
2. Open Arduino IDE and write the transmitter program.
3. Upload the program to the first Arduino board.
4.

Zigbee Communication Receiver Side:

1. Connect Zigbee receiver module: VCC → 5V, GND → GND, TX → Pin 2, RX → Pin 3.
2. Connect an LED to Pin 4 of Arduino.
3. Open Arduino IDE and write the receiver program.
4. Upload the program to the second Arduino board.
5. Power both Arduino boards.
6. Observe LED behavior on the receiver board to verify transmission**.**

CONNECTIONS:

Transmitter:

Arduino UNO Pin	Zigbee Module
VCC	**5V**
GND	**G**
2	**Tx**
3	**Rx**

Receiver:

Arduino UNO Pin	Zigbee Module	Arduino Development Board
-	**5V**	**5V**
-	**G**	**GND**
2	**Tx**	**-**
3	**Rx**	**-**
4	**-**	**LED1**

PROGRAM:

Transmitter

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2,3); // RX, TX
void setup() {
  mySerial.begin(9600);
  Serial.begin(9600);
}
void loop() {
  mySerial.write('A');
  Serial.println('A');
  delay(100);
  mySerial.write('B');
  Serial.println('B');
  delay(100);
}
```

Receiver

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2,3); // RX, TX
void setup() {
  mySerial.begin(9600);
  Serial.begin(9600);
  pinMode(4, OUTPUT);
}
void loop() {
  if(mySerial.available() > 0) {
  char data = mySerial.read();
  Serial.println(data);
    if(data == 'A') digitalWrite(4, HIGH);
    else if(data == 'B') digitalWrite(4, LOW);
  }
}
```

SAMPLE OUTPUT:

LED on the receiver board turns ON when transmitter sends 'A'.

LED turns OFF when transmitter sends 'B'.

RESULT:

Thus, the programs for different IoT communication methods were successfully written and executed in Embedded C using Arduino.

Ex.No:5	Introduction to Raspberry PI platform and python programming
Date:	

AIM:

To learn the basics of programming Raspberry Pi using MicroPython in Thonny IDE and to control LEDs and switches through GPIO.

HARDWARE & SOFTWARE REQUIRED:

S.No	Hardware/Software	Quantity
1	Raspberry Pi	1
2	LED	1
3	RGB LED	1
4	Push Button Switch	1
5	Jumper Wires	Few
6	USB Cable	1
7	Thonny IDE	1
8	Computer with Windows/Linux/Mac	1

Introduction to Raspberry Pi :

The Raspberry Pi is a compact and affordable microcontroller board developed by the Raspberry Pi Foundation. Building upon the success of the Raspberry Pi, the variant brings wireless connectivity to the table, making it an even more versatile platform for embedded projects. In this article, we will provide a comprehensive overview of the Raspberry Pi, highlighting its key features and capabilities.

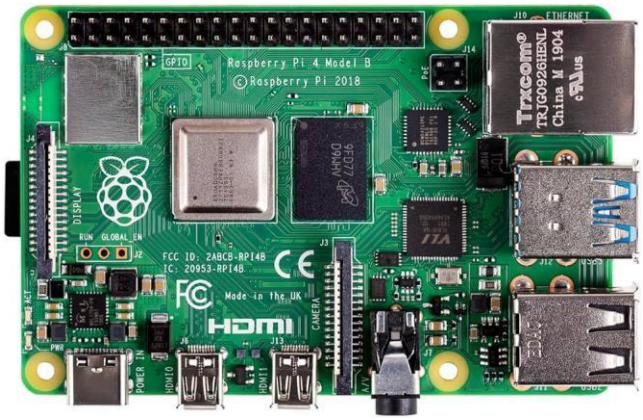
Features:

- RP2040 microcontroller with 2MB of flash memory
- On-board single-band 2.4GHz wireless interfaces (802.11n)
- Micro USB B port for power and data (and for reprogramming the flash)
- 40 pins 21mmx51mm ‘DIP’ style 1mm thick PCB with 0.1″ through-hole pins also with edge castellations
- Exposes 26 multi-function 3.3V general purpose I/O (GPIO)
- 23 GPIO are digital-only, with three also being ADC-capable
- Can be surface mounted as a module
- 3-pin ARM serial wire debug (SWD) port
- Simple yet highly flexible power supply architecture
- Various options for easily powering the unit from micro-USB, external supplies, or batteries
- High quality, low cost, high availability

- Comprehensive SDK, software examples, and documentation
- Dual-core Cortex M0+ at up to 133MHz
- On-chip PLL allows variable core frequency
- 264kByte multi-bank high-performance SRAM

Raspberry Pi:

The Raspberry Pi W is based on the RP2040 microcontroller, which was designed by Raspberry Pi in-house. It combines a powerful ARM Cortex-M0+ processor with built-in Wi-Fi connectivity, opening up a range of possibilities for IoT projects, remote monitoring, and wireless communication. The W retains the same form factor as the original , making it compatible with existing accessories and add-ons.



RP2040 Microcontroller:

At the core of the Raspberry Pi W is the RP2040 microcontroller. It features a dual-core ARM Cortex-M0+ processor running at 133MHz, providing ample processing power for a wide range of applications. The microcontroller also includes 264KB of SRAM, which is essential for storing and manipulating data during runtime. Additionally, the RP2040 incorporates 2MB of onboard flash memory for program storage, ensuring sufficient space for your code and firmware.

Wireless Connectivity:

The standout feature of the Raspberry Pi W is its built-in wireless connectivity. It includes an onboard Cypress CYW43455 Wi-Fi chip, which supports dual-band (2.4GHz and 5GHz) Wi-Fi 802.11b/g/n/ac. This allows the W to seamlessly connect to wireless networks, communicate with other devices, and access online services. The wireless capability opens up new avenues for IoT projects, remote monitoring and control, and real-time data exchange.

GPIO and Peripherals:

Similar to the Raspberry Pi , the W offers a generous number of GPIO pins, providing flexibility for interfacing with external components and peripherals. It features 26 GPIO pins, of which 3 are

analog inputs, and supports various protocols such as UART, SPI, I2C, and PWM. The W also includes onboard LED indicators and a micro-USB port for power and data connectivity.

MicroPython and C/C++ Programming:

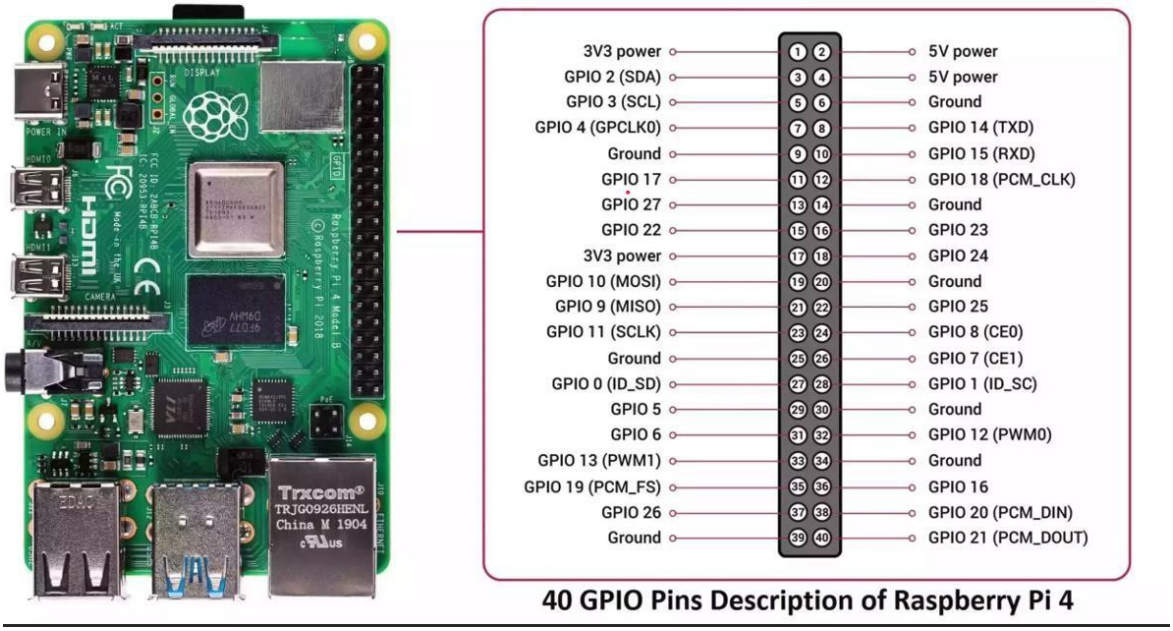
The Raspberry Pi W can be programmed using MicroPython, a beginner-friendly programming language that allows for rapid prototyping and development. MicroPython provides a simplified syntax and high-level abstractions, making it easy for newcomers to get started. Additionally, the W is compatible with C/C++ programming, allowing experienced developers to leverage the rich ecosystem of libraries and frameworks available.

Programmable Input/Output (PIO) State Machines:

One of the unique features of the RP2040 microcontroller is the inclusion of Programmable Input/Output (PIO) state machines. These state machines provide additional processing power and flexibility for handling real-time data and timing-critical applications. The PIO state machines can be programmed to interface with custom protocols, generate precise waveforms, and offload tasks from the main processor, enhancing the overall performance of the system.

Open-Source and Community Support

As with all Raspberry Pi products, the W benefits from the vibrant and supportive Raspberry Pi community. Raspberry Pi provides extensive documentation, including datasheets, pinout diagrams, and programming guides, to assist developers in understanding the board's capabilities. The community offers forums, online tutorials, and project repositories, allowing users to seek help, share knowledge, and collaborate on innovative projects. The Raspberry Pi W brings wireless connectivity to the popular Raspberry Pi microcontroller board. With its powerful RP2040 microcontroller, built-in Wi-Fi chip, extensive GPIO capabilities, and compatibility with MicroPython and C/C++ programming, the W offers a versatile and affordable platform for a wide range of embedded projects. Whether you are a beginner or an experienced developer, the Raspberry Pi W provides a user-friendly and flexible platform to bring your ideas to life and explore the exciting world of wireless IoT applications.



40 GPIO Pins Description of Raspberry Pi 4

INTRODUCTION TO PYTHON PROGRAMMING

What is MicroPython?

MicroPython is a Python 3 programming language re-implementation targeted for microcontrollers and embedded systems. MicroPython is very similar to regular Python. Apart from a few exceptions, the language features of Python are also available in MicroPython. The most significant difference between Python and MicroPython is that MicroPython was designed to work under constrained conditions.

Because of that, MicroPython does not come with the entire pack of standard libraries. It only includes a small subset of the Python standard libraries, but it includes modules to easily control and interact with the GPIOs, use Wi-Fi, and other communication protocols.

Thonny IDE:

Thonny is an open-source IDE which is used to write and upload MicroPython programs to different development boards such as Raspberry Pi , ESP32, and ESP8266. It is extremely interactive and easy to learn IDE as much as it is known as the beginner-friendly IDE for new programmers. With the help of Thonny, it becomes very easy to code in Micropython as it has a built-in debugger that helps to find any error in the program by debugging the script line by line.You can realize the popularity of Thonny IDE from this that it comes pre-installed in Raspian OS which is an operating system for a Raspberry Pi. It is available to install on r Windows, Linux, and Mac OS.

Installing Thonny IDE – Windows PC

1. Go to the official Thonny website:
   🔗 https://thonny.org
2. Download the Windows version of Thonny.
   Choose the correct installer (usually labeled “Windows installer (64-bit)”).
   Wait a few seconds for the download to complete.
3. Run the installer (.exe) file.
   Locate the downloaded file (e.g., thonny-xxx.exe).
   Double-click it to start installation.

4. Follow the Installation Wizard.
   Click “Next” through the setup steps.
   Accept the license agreement if prompted.
   Choose the default installation options unless you need a custom setup.
5. Finish Installation.
   Click “Finish” once the installation is complete.
   You can now launch Thonny from your Start Menu or desktop shortcut.

1. LED Blink:
CONNECTIONS:

Raspberry Pi Pin	Raspberry Pi Development Board
GP16	LED

PROCEDURE:

1. Connect the LED to GP16 pin of Raspberry Pi and GND.
2. Open Thonny IDE and connect to the Raspberry Pi board.
3. Write the LED blink program in MicroPython.
4. Run the program. Observe the LED blinking at 1-second intervals.

PROGRAM

```
from machine import Pin

import time

LED = Pin(16, Pin.OUT)

while True:

   LED.value(1)

   time.sleep(1)

   LED.value(0)

   time.sleep(1)
```

SAMPLE OUTPUT:

LED blinked successfully at 1-second intervals.

2. RGB LED CONTROL:
CONNECTIONS:

Raspberry Pi Pin	Raspberry Pi Development Board (RGB)
GP16	R
GP17	G
GP18	B
GND	COM

PROCEDURE:

1. Connect the RGB LED to GP16, GP17, GP18, and GND as per the connection table.
2. Write the RGB LED control program in Thonny.
3. Run the program and observe the LED changing colors sequentially.

PROGRAM

```
from machine import Pin
from time import sleep, sleep_ms
r = Pin(16, Pin.OUT)
y = Pin(17, Pin.OUT)
g = Pin(18, Pin.OUT)
while True:
    r.value(1)
    sleep_ms(1000)
    r.value(0)
    sleep_ms(1000)
    y.value(1)
    sleep(1)
    y.value(0)
    sleep(1)
    g.value(1)
    sleep(1)
    g.value(0)
    sleep(1)
```

SAMPLE OUTPUT:

RGB LED changed colors sequentially.

3. Switch-Controlled LED:

CONNECTIONS:

Raspberry Pi Pin	Raspberry Pi Development Board
GP16	**LED**
GP15	**SW1**

PROCEDURE:

1. Connect a push button to GP15 and GND; LED to GP16.
2. Write the switch-controlled LED program in Thonny.
3. Run the program and press/release the switch to turn the LED ON/OFF.
4. Observe the LED behavior and check the serial output in Thonny.

PROGRAM:

```
from machine import Pin
from time import sleep
```

```
led = Pin(16, Pin.OUT)
sw = Pin(15, Pin.IN)
while True:
    bt = sw.value()
    if bt == True:

        print("LED ON")
        led.value(1)
        sleep(2)
        led.value(0)
        sleep(2)
    else:
        print("LED OFF")
        sleep(0.5)
```

SAMPLE OUTPUT:

LED turned ON/OFF with the push button as expected.

RESULT:

Thus, programs to control LEDs and switches through GPIO were successfully written and executed in Embedded C using Arduino.

Ex.No:6	Interfacing sensor with Raspberry PI
Date:	

AIM:

To interface the IR sensor and Ultrasonic sensor with Raspberry.

HARDWARE & SOFTWARE TOOLS REQUIRED:

S.No	Hardware & Software Requirements	Quantity
1	Thonny IDE	1
2	Raspberry Pi Development Board	1
3	Jumper Wires	few
4	Micro USB Cable	1
5	IR Sensor	1

PROCEDURE

1. Install Thonny IDE: Ensure that Thonny IDE is installed on your computer. This IDE will be used to write and upload MicroPython programs to the Raspberry Pi .
2. Connect the OUT pin of the IR sensor to GP15 (or any GPIO pin of your choice) on the Raspberry Pi.
3. Connect the VCC of the IR sensor to 5V and GND to GND on the Raspberry Pi.
4. Use Thonny IDE to write the MicroPython code for reading data from the IR sensor. The code should check the IR sensor output and activate a buzzer or perform any other action when an object is detected.
5. Upload the Code: Upload the program to the Raspberry Pi using Thonny IDE.
6. Test the Setup: Observe the behavior of the IR sensor. The output can be seen via the buzzer, LED, or through the serial monitor.

CONNECTIONS:

Raspberry Pi Pin	Raspberry Pi Development Board	IR Sensor Module
GP16	**BUZZER**	**-**
GP15	**-**	**OUT**
-	**5V**	**VCC**
-	**GND**	**GND**

PROGRAM:

```
from machine import Pin
import time
# Setup for IR Sensor
IR_sensor = Pin(15, Pin.IN)  # IR sensor input pin (GP15)
buzzer = Pin(16, Pin.OUT)    # Buzzer output pin (GP16)
while True:
    if IR_sensor.value() == 1:  # Check if IR sensor detects an object
        buzzer.value(1)  # Turn ON Buzzer when IR sensor detects an object
        print("Object Detected")
    else:
        buzzer.value(0)  # Turn OFF Buzzer when no object is detected
        print("No Object Detected")
    time.sleep(0.1)  # Delay between checks
```

SAMPLE OUTPUT:

If no object is near the IR sensor:

No Object Detected

When you place an object in front of the IR sensor:

Object Detected

RESULT:

Thus, the program for the IR sensor was successfully interfaced with the Raspberry Pi and executed using MicroPython in Thonny IDE

Ex.No:7	Communicate between Arduino and Raspberry PI using any wireless medium
Date:	

AIM:

To write and execute the program to Communicate between Arduino and Raspberry PI Using any wireless medium (Bluetooth)

HARDWARE & SOFTWARE TOOLS REQUIRED:

S.No	Hardware & Software Requirements	Quantity
1	Thonny IDE	1
2	Raspberry Pi Development Board	1
3	Arduino Uno Development Board	1
4	Jumper Wires	few
5	Micro USB Cable	1
6	Bluetooth Module	2

MASTER

PROCEDURE

1. Connect the TX pin of the Bluetooth module to Pin 2 (RX) on the Arduino.Connect the RX pin of the Bluetooth module to Pin 3 (TX) on the Arduino.
2. Connect the VCC pin of the Bluetooth module to the 5V pin on the Arduino.Connect the GND pin of the Bluetooth module to the GND pin on the Arduino.
3. Open the Arduino IDE on your computer.Select the correct board (Arduino Uno) and the correct port under the Tools menu.
4. Include the necessary SoftwareSerial library to enable communication through the Bluetooth module.Initialize SoftwareSerial for Bluetooth communication on pins 2 (RX) and 3 (TX).
5. In the setup() function, start the Serial communication at 9600 baud for the Bluetooth module.
6. In the loop(), write data ('A' and 'B') using the mySerial.write() function to communicate with the Raspberry Pi every second.
7. Click the Upload button in the Arduino IDE to upload the program to the Arduino board.
8. The Arduino will now continuously send the characters 'A' and 'B' to the Raspberry Pi via Bluetooth.

CONNECTIONS:

Arduino UNO Pin	Arduino Development Board	Bluetooth Module
2	**-**	**Tx**
3	**-**	**Rx**
-	**GND**	**GND**
-	**5V**	**5V**

PROGRAM:

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2, 3);  // RX, TX for Bluetooth communication
void setup() {
  mySerial.begin(9600);  // Start Bluetooth serial communication
}
void loop() {
  mySerial.write('A');  // Send 'A' command to  (LED ON)
  delay(1000);          // Wait for 1 second
  mySerial.write('B');  // Send 'B' command to  (LED OFF)
  delay(1000);          // Wait for 1 second
}
```

SLAVE

PROCEDURE:

1. Connect the TX pin of the Bluetooth module to GP0 (RX) on the Raspberry Pi .Connect the RX pin of the Bluetooth module to GP1 (TX) on the Raspberry Pi .
2. Connect the VCC pin of the Bluetooth module to the 5V pin on the Raspberry Pi .Connect the GND pin of the Bluetooth module to the GND pin on the Raspberry Pi .
3. Open Thonny IDE on your computer.Select the MicroPython (Raspberry Pi) interpreter from the Tools > Options > Interpreter menu.
4. Initialize UART communication with the Bluetooth module at 9600 baud. Define the LED pin and set it as an output. In the while True loop, check if the UART has received any data using uart.any().
5. If data is received, read it and check if it's the command 'A' or 'B'.If the command is 'A', turn the LED ON. if the command is 'B', turn the LED OFF.Also, send a confirmation message back to the Arduino.
6. After writing the program, save it and run it in Thonny.The Raspberry Pi will start listening for Bluetooth commands from the Arduino.

CONNECTIONS:

Raspberry Pi Pin	Raspberry Pi Development Board	Bluetooth Module
GP16	**LED**	**-**
VCC	**-**	**+5V**
GND	**-**	**GND**
GP1	**-**	**Tx**
GP0	**-**	**Rx**

PROGRAM:

```
from machine import Pin, UART
uart = UART(0, 9600)
led = Pin(16, Pin.OUT
while True:
    if uart.any() > 0:
        data = uart.read
        print(data)
        if "A" in data:
            led.value(1)
```

```
        uart.write('LED on \n')

    elif "B" in data:
        led.value(0)
        uart.write('LED off \n')
```

SAMPLE OUTPUT:

```
The command is 'A', turn the LED ON.
The command is 'B', turn the LED OFF
```

RESULT:

Thus, the program to enable communication between Arduino and Raspberry Pi using Bluetooth as a wireless medium was successfully interfaced with the Raspberry Pi and executed using MicroPython in Thonny IDE.

Ex.No:8	Setup a cloud platform to log the data and Log Data using Raspberry PI and upload to the cloud platform
Date:	

AIM:

To set up a cloud platform to log the data and upload data using Raspberry Pi to the cloud platform.

HARDWARE & SOFTWARE TOOLS REQUIRED:

S.No.	Hardware & Software Requirements	Quantity
1	Blynk Platform	1
2	Thonny IDE	1
3	Raspberry Pi Development Board	few
4	Jumper Wires	1
5	Micro USB Cable	1

CLOUD PLATFORM-BLYNK:

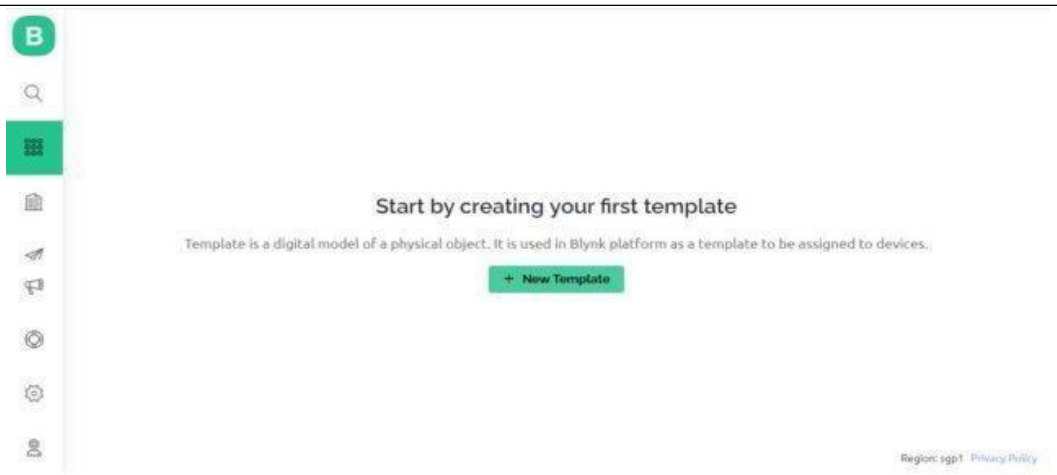
Blynk is a smart platform that allows users to create their Internet of Things applications without the need for coding or electronics knowledge. It is based on the idea of physical programming & provides a platform to create and control devices where users can connect physical devices to the Internet and control them using a mobile app.

Setting up Blynk 2.0 Application

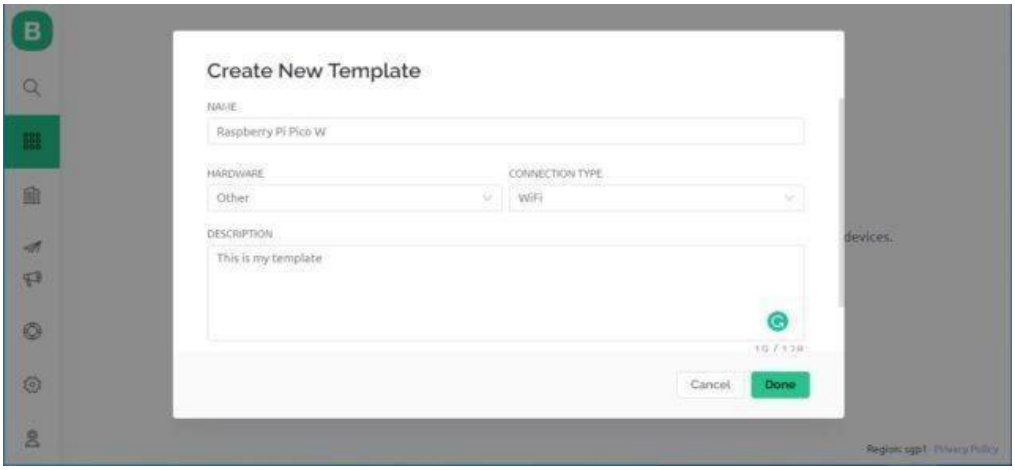
To control the LED using Blynk and Raspberry Pi , you need to create a Blynk project and set up a dashboard in the mobile or web application. Here's how you can set up the dashboard:

Step 1: Visit blynk.cloud and create a Blynk account on the Blynk website. Or you can simply sign in using the registered Email ID.

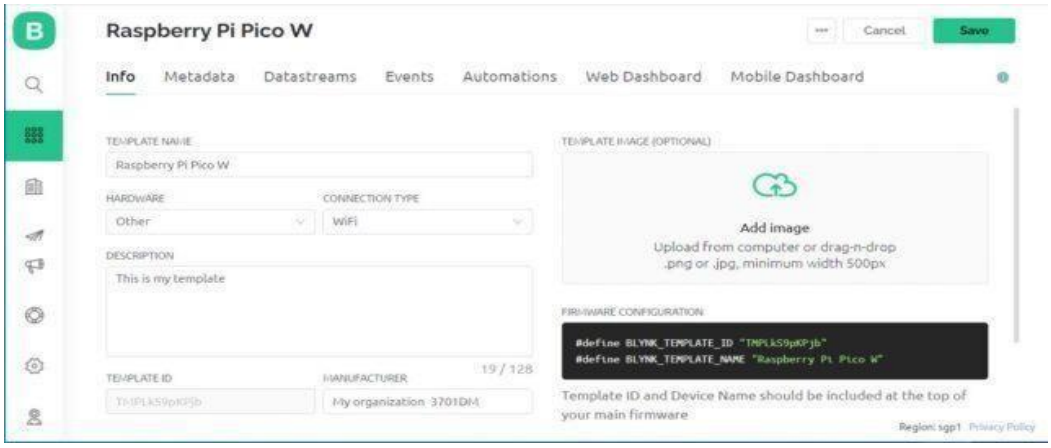
Step 2: Click on +New Template.



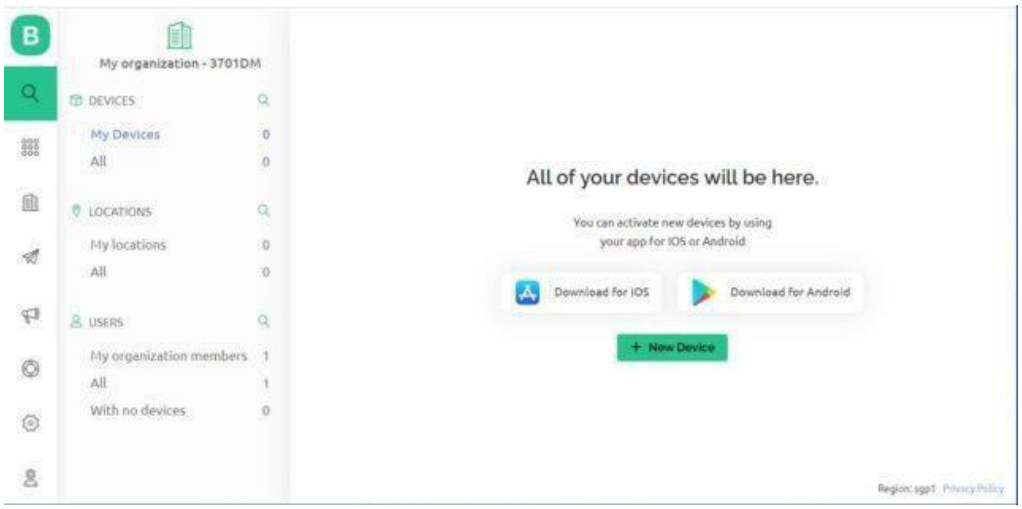
Step 3: Give any name to the Template such as Raspberry Pi W. Select 'Hardware Type' as Other and 'Connection Type' as WiFi.



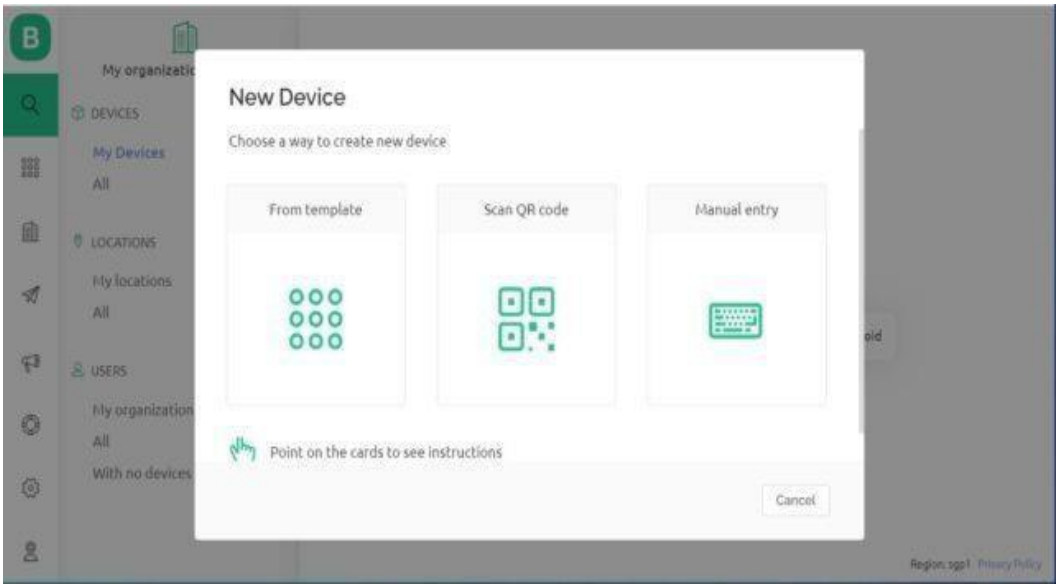
So a template will be created now.



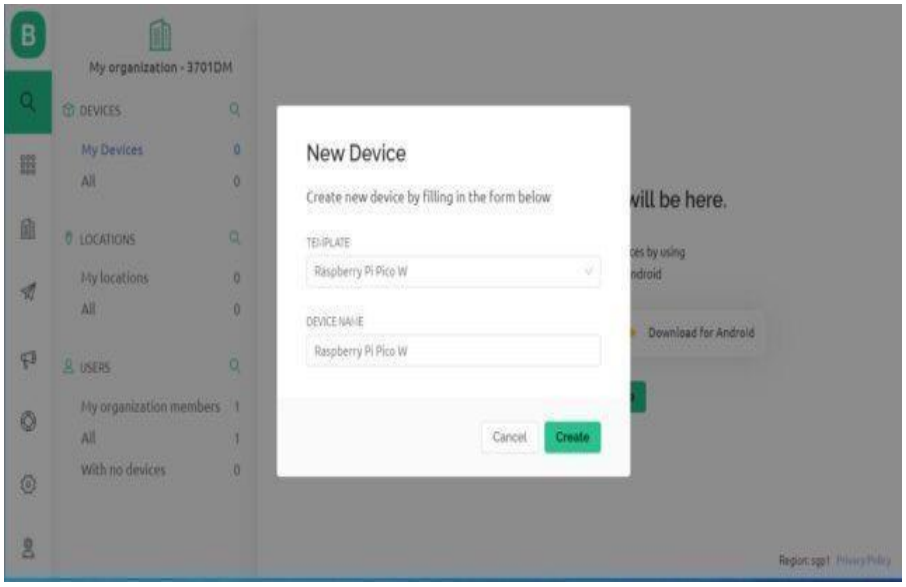
Step 4: Now we need to add a ‘New Device’ now.



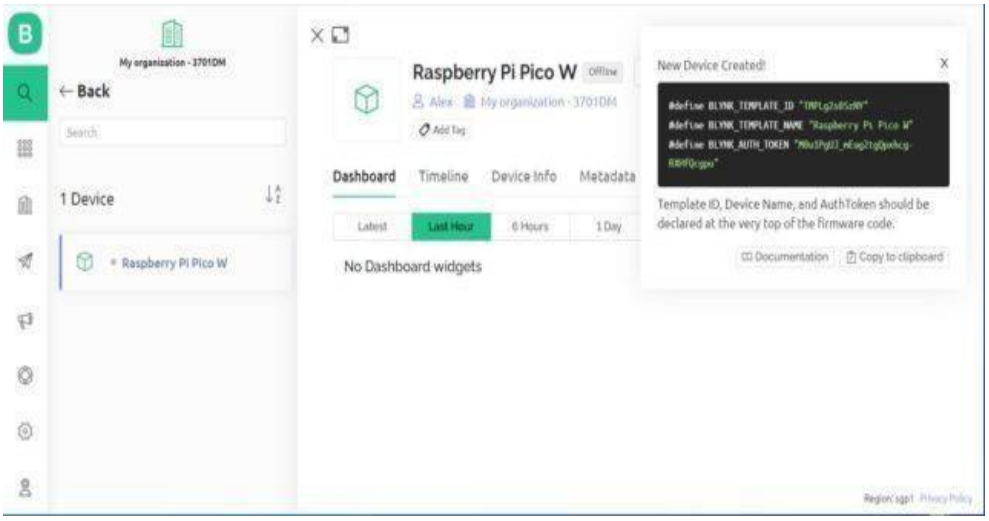
Select a New Device from ‘Template’.



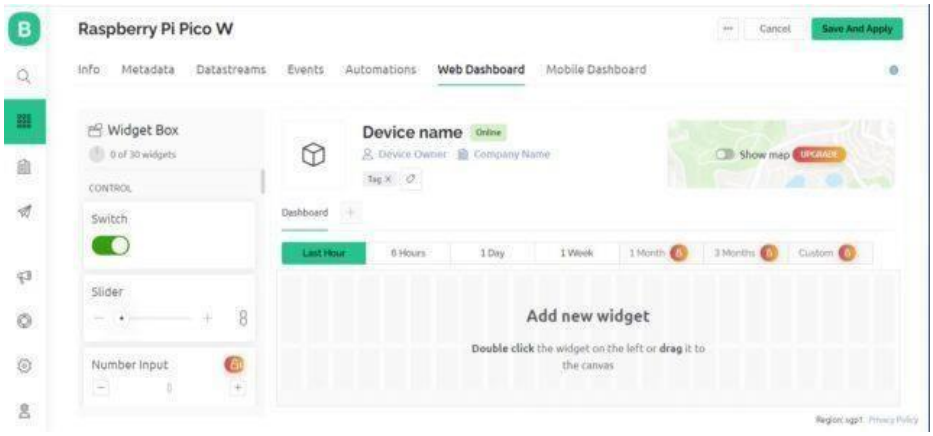
Select the device from a template that you created earlier and also give any name to the device. Click on Create.



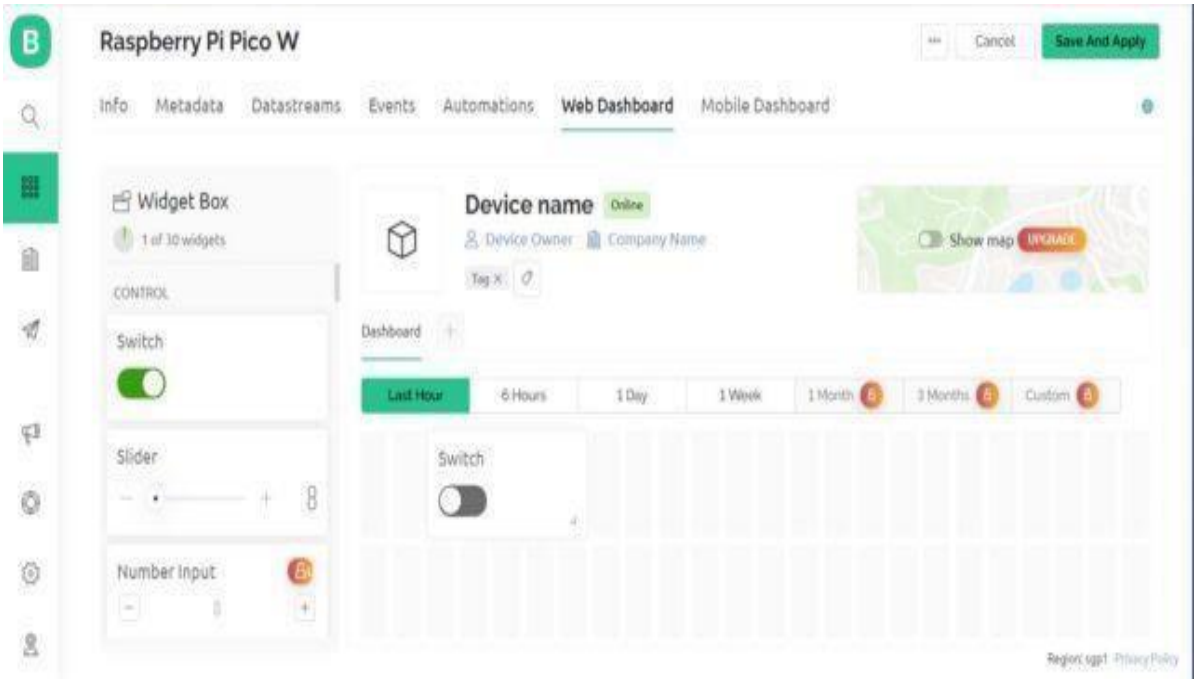
A new device will be created. You will find the Blynk Authentication Token Here. Copy it as it is necessary for the code.



Step 5: Now go to the dashboard and select ‘Web Dashboard’.



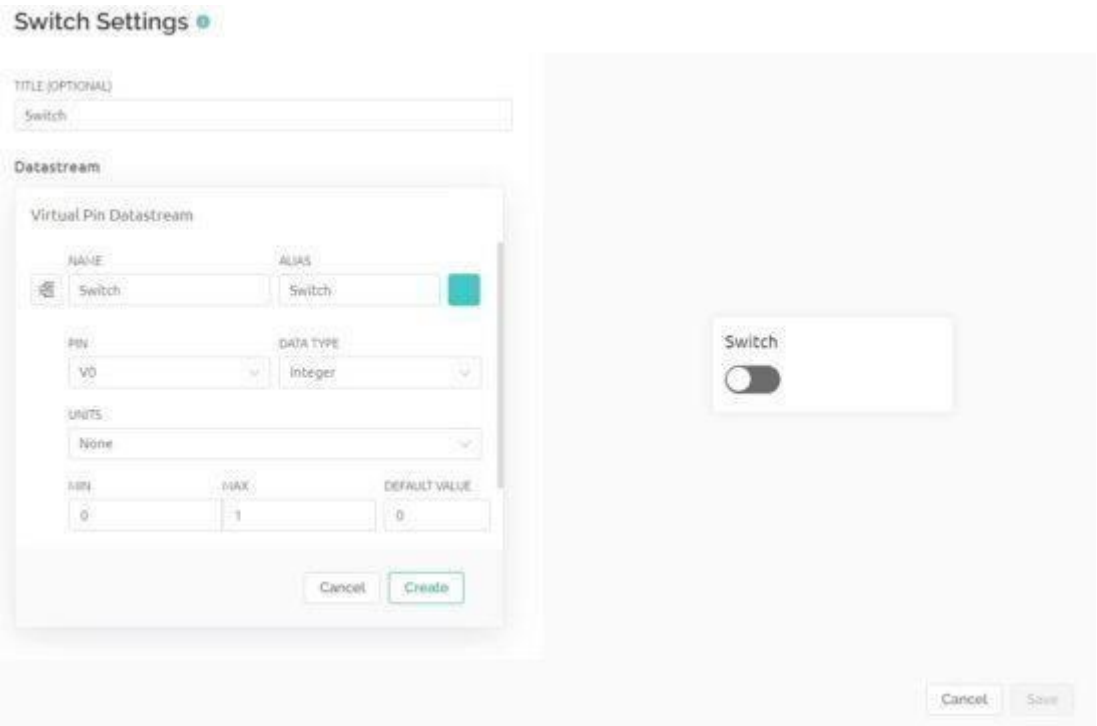
From the widget box drag a switch and place it on the dashboard screen.



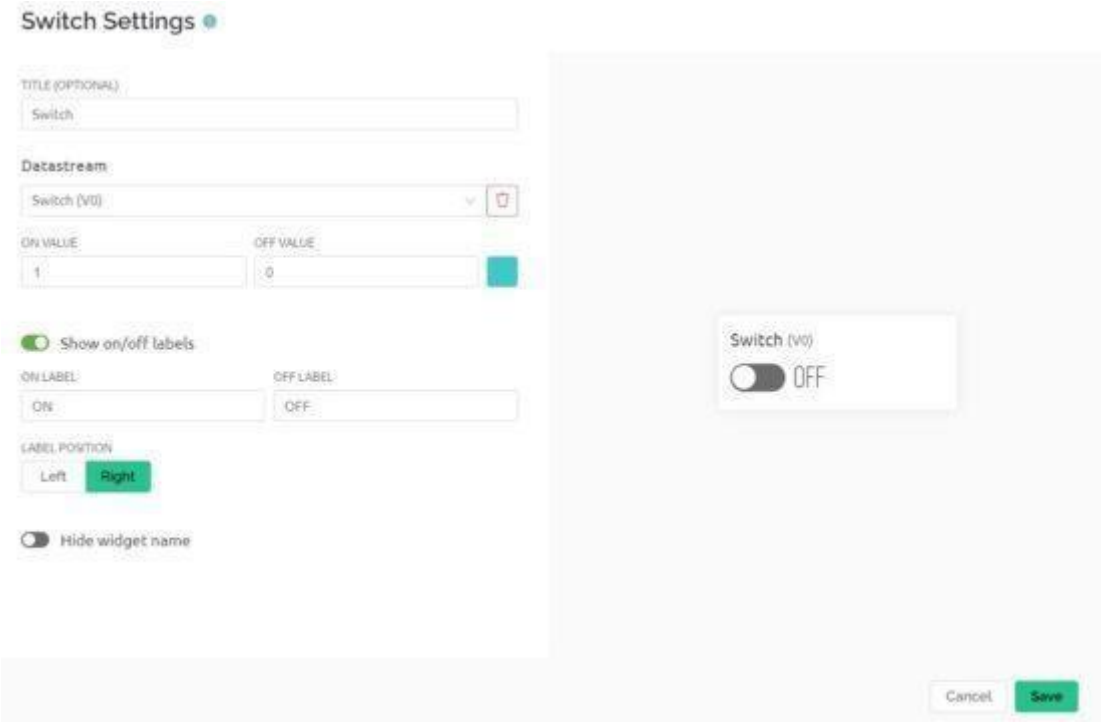
Step 6:

On the switch board click on Settings and here you need to set up the Switch. Give any title to it and Create Datastream as Virtual Pin

Configure the switch settings as per the image below and click on create.



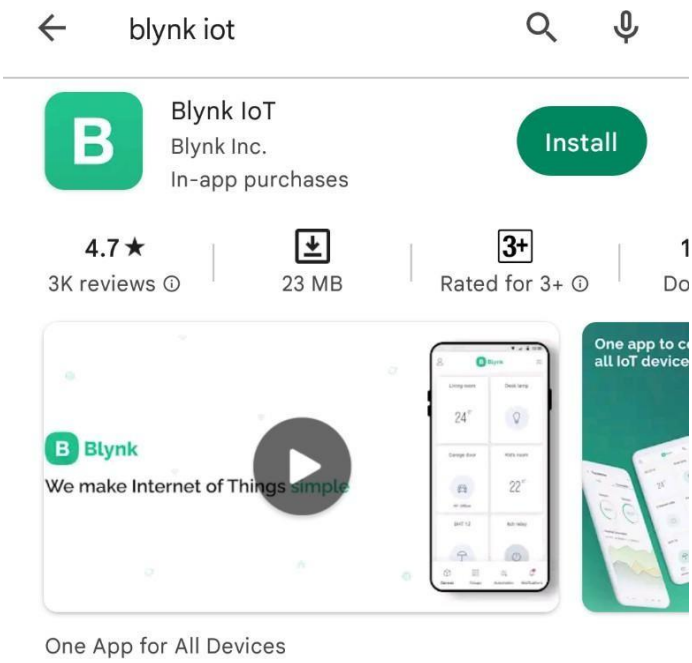
Configure the final steps again.



With this Blynk dashboard set up, you can now proceed to program the Raspberry Pi W board to control the LED.

Step 7:

To control the LED with a mobile App or Mobile Dashboard, you also need to setup the Mobile Phone Dashboard. The process is similarly explained above.



Install the Blynk app on your smartphone The Blynk app is available for iOS and Android. Download and install the app on your smartphone. then need to set up both the Mobile App and the Mobile Dashboard in order to control the LED with a mobile device. The process is explained above.

1. Open Google Play Store App on an android phone
2. Open Blynk.App
3. Log In to your account (using the same email and password)
4. Switch to Developer Mode
5. Find the “Raspberry Pi ” template we created on the web and tap on it
6. Tap on the “Raspberry Pi” template (this template automatically comes because we created it on our dashboard).
7. tap on plus icon on the left-right side of the window
8. Add one button Switch
9. Now We Successfully Created an android template
10. it will work similarly to a web dashboard template

PROCEDURE

1. First, connect the I2C LCD module and the temperature sensor to the appropriate pins on the Raspberry Pi , ensuring correct wiring for power, ground, SDA, and SCL.

2. Install the required libraries for the LCD (_i2c_lcd), Wi-Fi connection (network), and cloud platform integration (BlynkLib) in the Thonny IDE.
3. Write the program to read temperature data from the sensor, calculate Celsius and Fahrenheit values, and display the temperature on the LCD screen.
4. Set up Wi-Fi by connecting the Raspberry Pi to your local network and use the Blynk authentication token to connect to the Blynk cloud platform.
5. Run the program to continuously read temperature data, display it on the LCD, and upload the temperature readings to the Blynk cloud every 5 seconds for real-time monitoring.

CONNECTIONS:

Raspberry Pi Pin	Raspberry Pi Development Board	LCD Module
-	5V	VCC
-	GND	GND
GP0	-	SDA
GP1	-	SCL

PROGRAM:

```
from machine import Pin, I2C, ADC
from utime import sleep
from _i2c_lcd import I2cLcd
import network
import time
import BlynkLib
# === Wi-Fi & Blynk Settings ===
WIFI_SSID = "MyHomeWiFi"
WIFI_PASS = "mypassword123"
BLYNK_AUTH = "QwErTy1234AbCdEf"
# === Initialize ADC (on-board temperature sensor) ===
adc = ADC(4)
# === Initialize I2C for LCD ===
i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
I2C_ADDR = i2c.scan()[0]
lcd = I2cLcd(i2c, I2C_ADDR, 2, 16)
# === Connect to Wi-Fi ===
print("Connecting to Wi-Fi...")
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(WIFI_SSID, WIFI_PASS)
wait = 10
while wait > 0:
    if wlan.status() < 0 or wlan.status() >= 3:
        break
    wait -= 1
    print("Waiting for connection...")
    time.sleep(1)
if wlan.status() != 3:
    raise RuntimeError("Wi-Fi connection failed")
else:
```

```
    print("Connected")
    ip = wlan.ifconfig()[0]
    print("IP:", ip)
# === Initialize Blynk ===
print("Connecting to Blynk...")
blynk = BlynkLib.Blynk(BLYNK_AUTH)
lcd.clear()
lcd.putstr("System Ready")
# === Main Loop ===
while True:
    # Read temperature sensor (onboard)
    ADC_voltage = adc.read_u16() * (3.3 / 65535)
    temperature_c = 27 - (ADC_voltage - 0.706) / 0.001721
    temperature_f = 32 + (1.8 * temperature_c)
    print("Temp: {:.2f}°C / {:.2f}°F".format(temperature_c, temperature_f))
    # Display on LCD
    lcd.clear()
    lcd.move_to(0, 0)
    lcd.putstr("C: {:.2f}".format(temperature_c))
    lcd.move_to(0, 1)
    lcd.putstr("F: {:.2f}".format(temperature_f))
    # Send to Blynk
    blynk.virtual_write(3, temperature_c)
    blynk.virtual_write(4,  temperature_f)
    blynk.run()
    sleep(5)
```

SAMPLE OUTPUT:

```
Connecting to Wi-Fi...
Waiting for connection...
Waiting for connection...
Connected
IP: 192.168.1.102
Connecting to Blynk...
System Ready
Temp: 28.45°C / 83.21°F
Temp: 28.38°C / 83.08°F
Temp: 28.42°C / 83.15°F
Temp: 28.40°C / 83.12°F
```

RESULT:

Thus, the program to log data using Raspberry Pi and upload it to the cloud platform (Blynk) for remote monitoring was successfully executed using MicroPython in the Thonny IDE.

Ex.No:9	Design an IOT based system
Date:	

AIM:

To design and implement a Smart Home Automation System using the Raspberry Pi and Blynk IoT platform, allowing remote control of home appliances such as an LED or relay through the internet.

HARDWARE & SOFTWARE REQUIREMENTS

S.No	Hardware / Software	Quantity
1	Thonny IDE	1
2	Raspberry Pi Development Board	1
3	Jumper Wires	As required
4	Micro USB Cable	1
5	LED or Relay Module	1

PROCEDURE

1. Install Thonny IDE on your computer (if not already installed).
2. Connect the Raspberry Pi board to your computer via a micro-USB cable.
3. Set up the in Thonny:
    - Go to Tools → Options → Interpreter
    - Select MicroPython (Raspberry Pi)
    - Choose the correct COM port
4. Connect the hardware:
    - Connect an LED or Relay to GPIO pin GP16.
    - Ensure correct wiring for power (3.3V) and ground (GND).
5. Configure the Blynk app or web dashboard:
    - Create a new Blynk project.
    - Add a Button Widget linked to Virtual Pin V0.
    - Copy the Auth Token from Blynk.
6. Write and upload the Python program in Thonny.
7. Run the program and observe that pressing the button on the Blynk app toggles the LED or relay on the hardware.

CONNCTION:

Raspberry Pi Pin	Component
GP16	LED (or Relay Module Input)
GND	GND of LED/Relay
3.3V	Power Supply for Circuit

PROGRAM:

```
import time
import network
import BlynkLib
from machine import Pin
# === Wi-Fi and Blynk Settings ===
WIFI_SSID = "MyHomeWiFi"
WIFI_PASS = "mypassword123"
BLYNK_AUTH = "QwErTy1234AbCdEf"
# === Initialize LED Pin ===
led = Pin(16, Pin.OUT)
# === Connect to Wi-Fi ===
print("Connecting to Wi-Fi...")
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(WIFI_SSID, WIFI_PASS)
wait = 10
while wait > 0:
    if wlan.status() < 0 or wlan.status() >= 3:
        break
    wait -= 1
    print("Waiting for connection...")
    time.sleep(1)
if wlan.status() != 3:
    raise RuntimeError("Network connection failed")
else:
    print("Connected")
    ip = wlan.ifconfig()[0]
    print("IP Address:", ip)
# === Initialize Blynk ===
print("Connecting to Blynk...")
blynk = BlynkLib.Blynk(BLYNK_AUTH)
# === Virtual Pin Handler for LED Control (V0) ===
@blynk.on("V0")
def v0_write_handler(value):
    if int(value[0]) == 1:
        led.value(1)
        print("LED ON")
    else:
        led.value(0)
        print("LED OFF")
# === Main Loop ===
while True:
    blynk.run()
```

SAMPLE OUTPUT:

```
Connecting to Wi-Fi...
Waiting for connection...
Connected
IP Address: 192.168.1.102
Connecting to Blynk...
LED ON
LED OFF
LED ON
```

RESULT:

The Smart Home Automation System was successfully designed and implemented using the Raspberry Pi and Blynk IoT platform.